R47/C47 Matrix and Vector Menus including Vectors 2D & 3D

R47/C47 Matrix and Vector Introduction

A scalar is a single number. An ordered list of single numbers is a vector $\bar{v}$. A rectangular arrangement of i rows and j columns of single numbers is a matrix M. A vector is the special case where one of those dimensions is 1. The single number statement here includes complex numbers.

The R47/C47 supports matrices of arbitrary dimension, with 1×2, 2×1, 1×3 or 3×1 matrixes automatically recognised and handled as 2D vectors ($\bar{v}_2$) or 3D vectors ($\bar{v}_3$) respectively. Any single-row or single-column matrix is treated as a generic vector V. Both real and complex matrices M and vectors V are supported throughout, but 2D and 3D vectors are defined as real only.

Matrix addition and subtraction require identical dimensions. Multiplying an i×n matrix by an n×j matrix yields an i×j result. Scalar operations like LOG, SIN, |x| and many more apply element-wise.

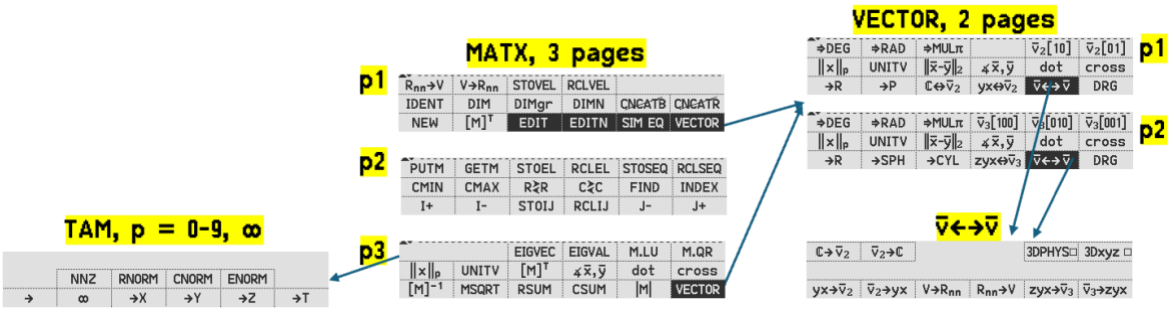
For square M, the determinant |M|, transpose M^T, and inverse M^{-1} are defined; M^{-1} exists only when $|M| \neq 0$. Division of M by a square matrix B is multiplication by B^{-1}. Two factorisations are available: M.LU returns the upper triangular factor U with partial pivoting applied, and M.QR returns the upper triangular factor R.

Solving $M\bar{x} = \bar{a}$ yields $\bar{x} = M^{-1}\,\bar{a}$, unique when M is square and non-singular.

For a square M, eigenvalue λ and eigenvector $\bar{v}$ satisfy $M\bar{v} = \lambda\bar{v}$. EIGVAL returns eigenvalues as roots of $\det(M - \lambda I) = 0$, which may be real or complex regardless of whether M is real. EIGVEC returns the eigenvectors; collecting these as columns of V gives $V^{-1}MV = \Lambda$, diagonal with eigenvalues on the main diagonal, when M is diagonalisable.

Several norms are available for any matrix or vector in x. The general p-norm is $\|x\|_p = (\Sigma|x_i|^p)^{(1/p)}$, supported for integer p from 1 to 9, and ∞, also including 0 for convenience although NNZ is not a norm. The practically important related cases are $\|x\|_1$ (maximum column sum of absolute values), $\|x\|_2$ the Frobenius norm $\sqrt{(\Sigma x_{ij}^2)}$ which reduces to the Euclidean vector magnitude, and $\|x\|_\infty$ the row norm, being the maximum row sum of absolute element values.

For 2D and 3D vectors $\bar{v}$, the R47/C47 provides a function set covering coordinate conversions, norms, inter-vector distance and angle, and cross product. Dot product is available for any conforming matrices. The axis labels follow the ijk convention (optionally xyz). All vectors are stored in rectangular form; polar, cylindrical and spherical representations are display modes only.



Left:

The matrix and vector menu structure for R47/C47

Matrix and Vector functions

The matrix functions available are:

- **RSUM**

  Returns a row vector containing the sum of each row of x.

- **CSUM**

  Returns a column vector containing the sum of each column of x.

- **NNZ** $\|x\|_0$

  Sparsity count (number of non-zero elements). Count of non-zero elements in vector and complete matrix. Scalar.

- **RNORM** $\|x\|_\infty = \max_{1 \le i \le m} \sum_{j=1}^{n} |\, a_{ij} \,|$

  Maximum norm. For a matrix, the maximum row sum of absolute element values; for a row vector, the sum of absolute values; for a column vector, the largest absolute element.

- **CNORM** $\|x\|_1 = \max_{1 \le j \le n} \sum_{i=1}^{m} |\, a_{ij} \,|$

  L^1 norm, a.k.a. Manhattan norm. For a matrix, the maximum column sum of absolute element values; for a column vector, the sum of absolute values; for a row vector, the largest absolute element.

- **ENORM** $\|x\|_2 \ (\sum_{i=1}^{n} |\, x_i \,|^2)^{1/2}$

  For a vector, Euclidean magnitude. For a matrix, Frobenius norm. $\Sigma|x_{ij}|^2$.

- $\|x\|_p \quad \left(\sum_{j=1}^{n} \sum_{i=1}^{m} |\, x_{ij} \,|^p\right)^{1/p}$ p = 3 to 9 (for 2, see above)

  General p-norm for integer p >= 2.

- **M.LU**

  Decomposes M into lower and upper triangular factors; underlies inversion and linear solving.

- **M.QR**

  Decomposes M into an orthogonal matrix times an upper triangular matrix; underlies eigenvalue computation.

- **DOT**

  Dot product of two conforming matrices; returns the scalar $\Sigma a_{ij} b_{ij}$.

- **CROSS**

  Cross product of two 2D or 3D vectors; returns a vector orthogonal to both operands.

- **$[M]^{-1}$**

  Matrix inverse; exists only when $|M| \neq 0$.

- **|M|**

Determinant of square matrix M.

- **UNITV**

  Returns the unit vector/matrix in x; equivalent to $\mathbf{x}/\|\mathbf{x}\|_2$, x being either M or $\bar{\mathbf{v}}$. The resulting unit vector will have $\|\mathbf{x}\|_2 = 1$.

- **SIM EQ**

  Solves $[A] \cdot \vec{X} = \vec{B}$ for the unknown $\vec{X}$ given coefficient matrix [A] and constant vector $\vec{B}$.

Matrices and Vectors — Worked Examples

A =	[1 -2 3 4 0 -1 2 3 5]
B =	[1 2 3]^T
X =	[1 2 3]
Y =	[3 1 2]
Row and Column Operations	
RSUM of A	[2 3 10]^T
RSUM of B	[1 2 3]^T
RSUM of X	[6]
CSUM of A	[7 1 7]
CSUM of B	[6]
CSUM of X	[1 2 3]
Norms	
NNZ $\|x\|_0$ of A	= 8 (one zero at row 2, col 2)
NNZ $\|x\|_0$ of B and X	= 3
RNORM $\|x\|_\infty$ of A	row 1: 1+2+3 = 6 row 2: 4+0+1 = 5 row 3: 2+3+5 = 10 → RNORM = 10

RNORM $\|x\|_\infty$ of B (column vector)	max(1, 2, 3) = 3 (each row one element)
RNORM $\|x\|_\infty$ of X (row vector)	1+2+3 = 6 (one row, all elements)
CNORM $\|x\|_1$ of A	col 1: 1+4+2 = 7 col 2: 2+0+3 = 5 col 3: 3+1+5 = 9 → CNORM = 9
CNORM $\|x\|_1$ of B (column vector)	1+2+3 = 6 (one col, all elements)
CNORM $\|x\|_1$ of X (row vector)	max(1, 2, 3) = 3 (each column one element)
ENORM $\|x\|_2$ of A	$\sqrt{(1+4+9+16+0+1+4+9+25)} = \sqrt{69} \approx 8.307$
ENORM $\|x\|_2$ of B and X	$\sqrt{(1+4+9)} = \sqrt{14} \approx 3.742$
$\|x\|_3$ of A	(1+8+27+64+0+1+8+27+125)^(1/3) = 261^(1/3) ≈ 6.391

Decompositions

M.LU of A (M.LU of A (upper triangular U, partial pivoting applied))	[4 0 -1 0 3 5.5 0 0 6.917]
M.QR of A (M.QR of A (upper triangular R, partial pivoting applied)	[-4.58 -0.87 -1.97 0 -3.50 -2.08 0 0 5.18]

Vector Products

DOT of X and Y	3×1 + 1×2 + 2×3 = 3+2+6 = 11
CROSS of X and Y	(1×3−2×2)i + (2×1−3×3)j + (3×2−1×1)k = [-1 -7 5]

Matrix Operations

|A| determinant of A	1×(0×5−(−1)×3) − (−2)×(4×5−(−1)×2) + 3×(4×3−0×2) = 3 + 44 + 36 = 83
$[A]^{-1}$ inverse of A	= (1/83) × [3 19 2 -22 -1 13 12 -7 8] ≈ [0.036 0.229 0.024

	-0.265 -0.012 0.157 0.145 -0.084 0.096]
Unit Vector and Linear System	
UNITV of B = $B/\|B\|_2$	= [$1/\sqrt{14}$ $2/\sqrt{14}$ $3/\sqrt{14}$]^T ≈ [0.267 0.535 0.802]^T
SIM EQ solves A·X = B A in Mat_A; B in Mat_B; result => Mat_X or use RCL A [M]^-1 RCL B ×	= (1/83)×[47 15 22]^T ≈ [0.566 0.181 0.265]^T
More examples	
Change variables:	B = [1 2 3]^T X = [4 5 6] A = [1 2 3 C = [1 2 1 4 5 6 0 1 3 7 8 9] 2 0 1]
RSUM of A in x	[6, 15, 24]
CSUM of A in x	[12, 15, 18]
NNZ of A in x = $\|x\|_0$	9
RNORM of A in x = $\|x\|_\infty = \max_{1\le i\le m} \sum_{j=1}^{n} \|a_{ij}\|$	max([6, 15, 24]) = 24
CNORM of A in x = $\|x\|_1 = \max_{1\le j\le n} \sum_{i=1}^{m} \|a_{ij}\|$	max([12, 15, 18]) = 18
ENORM of A in x = $\|x\|_2 = (\sum_{i=1}^{n} \|x_i\|^2)^{(1/2)}$	= sqrt($1^2+2^2+3^2+4^2+5^2+6^2+7^2+8^2+9^2$) = sqrt(285) ≈ 16.88
$\|x\|_p$ of A in x =$(\sum_{j=1}^{n} \sum_{i=1}^{m} \|x_{ij}\|^p)^{(1/p)}$ p=3	=(1+8+27+64+125+216+343+512+729)^(1/3) =(2025)^(1/3) ≈ 12.66
DOT of A in x and B in y	[1 2 3 [1*1 + 2*2 + 3*3 4 5 6 ● [1 2 3]T = 4*1 + 5*2 + 6*3 7 8 9] 7*1 + 8*2 + 9*3] = [14 32 50]^T

CROSS of B in x and B in y	= $[2*3-3*2,\ 3*1-1*3,\ 1*2-2*1]^T$ = $[0\ 0\ 0]^T$
$[M]^{-1}$ of C in x	= [1/11 -2/11 5/11 6/11 -1/11 -3/11 -2/11 4/11 1/11] use IRFRAC mode)
|M| of C in x	= 11
UNITV of B in x = $B/\|B\|_2$	= $[1/\sqrt{14},\ 2/\sqrt{14},\ 3/\sqrt{14}]$ ≈ [0.267, 0.535, 0.802]
SIM EQ: AX = B C in Mat_A; B in Mat_B; result Mat_X	X ≈ [1.09 -0.455 0.818]

Vectors, 2D & 3D functions

Vectors are fundamental objects in physics and engineering, yet scientific calculators have historically given them limited native support. This application note describes the R47/C47 vector implementation, which treats 2D and 3D vectors as first-class register objects with full polar, cylindrical and spherical display support.

The core design philosophy is clean: vectors are always stored in rectangular form. Polar, cylindrical and spherical representations are a display and entry convenience, not a separate storage format. This means all element-wise arithmetic operates on the underlying rectangular values and always returns a rectangular result, regardless of how the vector was entered or is currently displayed.

The implementation covers the complete set of vector operations one would expect: Euclidean norm, unit vector, dot and cross products, inter-vector distance and angle, and the full suite of coordinate conversions between rectangular, polar, cylindrical and spherical forms. Both 2D and 3D vectors are supported, with 2D polar and 3D spherical/cylindrical mode tagging carried through operations where it makes sense, for example:

- **[5 0.927^{r} 5]^{c}** 3D vector, tagged cylindrical, tagged as an angle of 0.927 radians
- **[3 4 5]** 3D vector, not tagged, the native underlying format of ijk or xyz visible

The axis labelling follows the conventional ijk notation used in engineering and physics, with an option to switch to xyz. Angle modes follow the calculator's active angle mode and can be converted freely using the usual double-arrow conversion keys.

Vector operations:

All vectors are stored as rectangular 1×2 or 1×3 matrices. Polar, cylindrical and spherical forms are display representations only, controlled by register tagging. Any matrix function will clear the polar tag and return the vector to the default rectangular display state.

Example: All element wise operations (+, -, ×, /, y^x, √x, etc.) are executed on the underlying rectangular coordinate values, regardless of the polar display status of the vector.

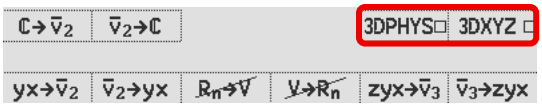
- **[3 4 5] 2 [×]** will result in **[6 8 10]**
- **[3 4 5] [→SPH] 2 [×]** will result in **[6 8 10]**
- **[5 0.927^{r} 5]^{c} [3 4 5] [+]** will result in **[6 8 10]**

- EDIT: Polar mode vector editing is not supported
- SHOW: No special vector or matrix SHOW display exists
- STO/RCL VEL1, 2, n: Vector element recalls underlying rectangular coordinates

Coordinate systems and notation

All vectors stored as rectangular [i j] (2D) or [i j k] (3D).

An alternative display using x & y, i.e. [x y] and [x y z] is available using the option flag [3DXYZ] in both the **v̄←→v̄** vector conversion menu and the system flags in CFLG menu.



Display modes tag the stored rectangular values; all element-wise operations act on the underlying rectangular components. Component ordering by display mode:

	3DXYZ clear			3DXYZ set		
Display mode	**Z**	**Y**	**X**	**Z**	**Y**	**X**
Cartesian 2D		i	j		x	y
Cartesian 3D	i	j	k	x	y	z
Polar 2D		r	θ_{ij}		r	θ_{xy}
Cylindrical 3D	r	θ_{ij}	k	r	θ_{xy}	z

Math Vectors	3DPHYS clear					
Spherical 3D	ρ	θ_{ij}	ϕ_k	ρ	θ_{xy}	ϕ_z
Physics Vectors	3DPHYS set					
Spherical 3D	ρ	ϕ_k	θ_{ij}	ρ	ϕ_z	θ_{xy}

Coordinate conversions (using the default [i j k] Math-style vector options):

Rectangular → Polar (2D): [i, j] → [r, θ_{ij}]

$r = \sqrt{(i^2 + j^2)}, \quad \theta_{ij} = \text{atan2}(j, i)$

Polar → Rectangular (2D): [r, θ_{ij}] → [i, j]

$i = r \cdot \cos(\theta_{ij}), \quad j = r \cdot \sin(\theta_{ij})$

Cartesian → Cylindrical (3D): [i, j, k] → [r, θ_{ij}, k]

$r = \sqrt{(i^2 + j^2)}, \quad \theta_{ij} = \text{atan2}(j, i), \quad k = k$

Cylindrical → Spherical (3D): [r, θ_{ij}, k] → [ρ, θ_{ij}, ϕ_k]

$\rho = \sqrt{(r^2 + k^2)}, \quad \theta_{ij} = \theta_{ij}, \quad \phi_k = \text{atan2}(r, k)$

Spherical → Cartesian (3D): [ρ, θ_{ij}, ϕ_k] → [i, j, k]

$i = \rho \cdot \sin(\phi_k) \cdot \cos(\theta_{ij}), \quad j = \rho \cdot \sin(\phi_k) \cdot \sin(\theta_{ij}), \quad k = \rho \cdot \cos(\phi_k)$

Not implemented in 2D and 3D:

Item	Feature	Possibility
Longpress ← (EDIT)	EDIT $\bar{v}_2$ or $\bar{v}_3$ POLAR	No editing of polar/cyl/sph values
STO/RCL VEL1, 2, nn	Store/Recall vector element	Accesses only the rectangular components
⇒DEG ⇒RAD ⇒MULπ DRG	[→P] conversion from rect to $\bar{v}_2$ or $\bar{v}_3$	No conversion from rectangular to polar with the Angle type
→R →P →R →SPH →CYL	Change to input polar directly from stack	No implied conversion from rectangular to polar with the polar type selection

Scalar operations from both 2D and 3D pages:

⇒DEG ⇒RAD ⇒MULπ Copied from MODE/TRG	Scalar numbers entered as angles and converted to other angle modes:	123 [→DEG] 123 [→DEG] [→RAD]	X: 123.° X: 2.147ʳ
DRG Copied from Keyboard/TRG	Angles entered as the current ADM; tagged angles converted to other angle modes:	[DEG] 123 [DRG] [RAD] 123 [DRG] [DRG]	X: 123.° X: 123.ʳ X: 7830.423ᵍ

2D vector operations:

⇒DEG	⇒RAD	⇒MULπ		$\bar{v}_2$[10]	$\bar{v}_2$[01]
$\|x\|_p$	$\|\bar{x}-\bar{y}\|_2$	$\angle\bar{x},\bar{y}$	UNITV	dot	cross
→R	→P	ℂ↔$\bar{v}_2$	yx↔$\bar{v}_2$	$\bar{v}$↔$\bar{v}$	DRG

2D: C47

⇒DEG	⇒RAD	⇒MULπ		$\bar{v}_2$[10]	$\bar{v}_2$[01]
$\|x\|_p$	$\|\bar{x}-\bar{y}\|_2$	$\angle\bar{x},\bar{y}$	UNITV	dot	cross
→R	→P	ℂ↔$\bar{v}_2$	yx↔$\bar{v}_2$	$\bar{v}$↔$\bar{v}$	CLSTK

2D: R47

All examples using default 3DXYZ clear:

yx↔$\bar{v}_2$	Creating 2D vector; to and from YX on stack:	4 ENTER 1 [yx↔$\bar{v}_2$] 4 ENTER 1 [DRG][yx↔$\bar{v}_2$] [4. 1.] [yx↔$\bar{v}_2$] [4. 1.°]ᵖ[yx↔$\bar{v}_2$]	X: [ij] [4. 1.] X:[rθij] [4. 1.°]ᵖ X: 4. Y:1. X: 4. Y:1.°
ℂ↔$\bar{v}_2$	Creating 2D vector; - to and from stack: - to and from complex no in x:	1[i]2 [ℂ↔$\bar{v}_2$] [DEG] 1 [$i_\odot$] 2 [ℂ↔$\bar{v}_2$] [1 4] [ℂ↔$\bar{v}_2$]	X:[ij] [1. 2.] X:[rθij] [1. 2.°]ᵖ X: 1.+i4.
→R →P Standard g-shifted keys	Rectangular <> Polar commands:	3 ENTER 4 [yx↔$\bar{v}_2$] [→P] 2 [$i_\odot$] 60 [ℂ↔$\bar{v}_2$] [→R]	X:[rθij] [5., 53.13°]ᵖ X:[ij] [1. 1.732]
⇒DEG ⇒RAD ⇒MULπ DRG	Change $\bar{v}_2$ (polar) angle mode, e.g. to Multπ:	2 [$i_\odot$] 60 [ℂ↔$\bar{v}_2$] [⇒MULπ]	X: [2. 0.333ʳʳ]ᵖ
$\bar{v}_2$[10] $\bar{v}_2$[01]	Create a new unit vector of length 1:	$\bar{v}_2$[10] 2 * $\bar{v}_2$[01] 3 * + $\bar{v}_2$[10] 3 * $\bar{v}_2$[01] 2 * +	Y:[ij] [2. 3.] X:[ij] [3. 2.]
$\|x\|_p$ ENORM	Euclidean Norm of V in x	[3. 4.] ENORM	X: 5.
UNITV	Create equivalent unit length vector from M or $\bar{v}$ in x	[3. 2.] UNITV [0.83295 0.5547] [→P]	Y:[ij] [0.83295 0.5547] X:[rθij] [1. 33.690°]
$\|\bar{x}-\bar{y}\|_2$	Distance between M or $\bar{v}$ in x and y	[2. 3.][3. 2.] [$\bar{x}-\bar{y}$]	X: 1.4142
$\angle\bar{x},\bar{y}$	Angle between M or $\bar{v}$ in x and y	[2. 3.][3. 2.] $\angle\bar{v},\bar{v}$	X: 22.620°
dot	Dot product of M or $\bar{v}$ in x and y	[2. 3.][3. 2.] dot	X: 12
cross	Cross product of $\bar{v}$ in x and y	[2. 3.][3. 2.] cross	X:[ijk] [0. 0. -5]
$\bar{v}$↔$\bar{v}$ yx→$\bar{v}_2$ $\bar{v}_2$→yx	Single direction vector <> stack conversions:	1 ENTER 30° [yx→$\bar{v}_2$] [1., 30°]ᵖ [$\bar{v}_2$→yx]	[1., 30°]ᵖ Y: 1 X: 30°
$\bar{v}$↔$\bar{v}$ ℂ→$\bar{v}_2$ $\bar{v}_2$→ℂ	Single direction vector <> complex conversions:	1 [$i_\odot$] 30° [ℂ→$\bar{v}_2$] [1., 30°]ᵖ [$\bar{v}_2$→ℂ]	[1., 30°]ᵖ 1∠30°

3D vector operations:

→DEG	→RAD	→MULπ	$\bar{v}_3[100]$	$\bar{v}_3[010]$	$\bar{v}_3[001]$
$\|\|x\|\|_p$	$\|\|\bar{x}-\bar{y}\|\|_2$	$\angle\bar{x},\bar{y}$	UNITV	dot	cross
→R	→SPH	→CYL	zyx↔$\bar{v}_3$	$\bar{v}$↔$\bar{v}$	DRG

3D: C47

→DEG	→RAD	→MULπ	$\bar{v}_3[100]$	$\bar{v}_3[010]$	$\bar{v}_3[001]$
$\|\|x\|\|_p$	$\|\|\bar{x}-\bar{y}\|\|_2$	$\angle\bar{x},\bar{y}$	UNITV	dot	cross
→R	→SPH	→CYL	zyx↔$\bar{v}_3$	$\bar{v}$↔$\bar{v}$	CLSTK

3D: R47

All examples using default 3DXYZ clear, 3DPHYS clear:

zyx↔$\bar{v}_3$	Creating 3D vector; to and from YX on stack:	1 EXIT 2 EXIT 3 [zyx↷$\bar{v}_3$] 1 EXIT 2° EXIT 3° [zyx↷$\bar{v}_3$] 1 EXIT 2° EXIT 3 [zyx↷$\bar{v}_3$] [1. 2.° 3.°]S [zyx↷$\bar{v}_3$] [1. 2.° 3.]C [zyx↷$\bar{v}_3$] [1. 2. 3.] [zyx↷$\bar{v}_3$]	X:[ijk] [1. 2. 3.] X:[ρ θij ϕk] [1. 2.° 3.°]S X:[r θij k] [1. 2.° 3.]C Z: 1.; Y: 2.°; X: 3.° Z: 1.; Y: 2.°; X: 3. Z: 1.; Y: 2.; X: 3.
→R →SPH →CYL	Rectangular <> Spherical <> Cylindrical conversion ([→P] does [→SPH])	[1. 2.° 3.°]S [→R] [1. 30.° 1.]C [→SPH] [3.×√2 π/6r π/4r]S [→CYL] [8. 30° 3.] [→R] [3. 1. 0.] √x [→SPH]	X:[ijk][0.052 0.002 .999] X:[ρθij ϕk][√2 30.° 45.°]S X: [r θij k] [3. π/6r 3.]C X:[ijk] [4.×√3 4. 3.] X:[ρθij ϕk] [2. 30° 90°]S
→DEG →RAD →MULπ DRG	Change SPH or CYL vector angle mode, e.g. To Multπ:	[1. -1.57r 3.14r]S [→MULπ]	X:[ρθij ϕk][1. -0.5πr 1.πr]S
$\bar{v}_3[100]$ $\bar{v}_3[010]$ $\bar{v}_3[001]$	Create a new unit vector of length 1:	$\bar{v}_2[100]$ 2 × $\bar{v}_2[010]$ 3 × + $\bar{v}_2[010]$ 3 × $\bar{v}_2[001]$ 2 × +	X:[ijk] [2. 3. 0] X:[ijk] [0. 3. 2]
$\|\|x\|\|_p$ ENORM	Euclidean Norm of V in x	X:[ijk] [2. 3. 6.] ENORM	X: 7
UNITV	Create equivalent unit length vector from M or $\bar{v}$ in x	[3. 2. 1.] UNITV	X: [0.802 0.535 0.267]
$\|\|\bar{x}-\bar{y}\|\|_2$	Distance between M or $\bar{v}$ in x and y	[1. 2. 3.][3. 2. 1.] [$\bar{x}-\bar{y}$]	2.×√2
$\angle\bar{x},\bar{y}$	Angle between M or $\bar{v}$ in x and y	[1. 2. 3.][3. 2. 1.] [$\angle\bar{x},\bar{y}$]	0.775r
dot	Dot product of M or $\bar{v}$ in x and y	[1. 2. 3.][3. 2. 1.] [dot]	10
cross	Cross product of $\bar{v}$ in x and y	[1. 2. 3.][3. 2. 1.] [cross]	[-4. 8. -4]
$\bar{v}$↔$\bar{v}$ zyx→$\bar{v}_3$ $\bar{v}_3$→zyx	Single direction vector conversions	1 EXIT 2 EXIT 3 [zyx→$\bar{v}_3$] [1. 2.° 3.°]S [zyx→$\bar{v}_3$]	X:[ijk] [1. 2. 3.] Z: 1.; Y: 2.°; X: 3.°

2D and 3D Examples with formulas

Cartesian to Cylindrical

Input: $\mathbf{r} = 3\mathbf{i} + 4\mathbf{j} + 5\mathbf{k} = [3\ 4\ 5]$

Convert: $[x, y, z] \rightarrow [r, \theta, z]$ where $r = \sqrt{(x^2 + y^2)}$, $\theta = \text{atan2}(y/x)$, $z = z$

$r = \sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$ $\theta = \arctan(4/3) = 53.130°$ $z = 5$

>> Output: $[r, \theta_{ij}, z_k] = [5\ 53.130°\ 5]^C$

Cylindrical to Spherical

Input: $[r, \theta_{ij}, z_k] = [5\ 53.130°\ 5]^C$

Convert: $[r, \theta_{ij}, z_k] \rightarrow [\rho, \theta_{ij}, \phi_k]$ where $\rho = \sqrt{(r^2 + z^2)}$, $\theta = \theta$, $\phi = \text{atan2}(r/z)$

$\rho = \sqrt{(5^2 + 5^2)} = \sqrt{50} = 7.071$ $\theta = 53.130°$ $\phi = \arctan(5/5) = 45°$

>> Output: $[\rho, \theta_{ij}, \phi_k] = [7.071,\ 53.130°,\ 45°]^S$

Spherical to Cartesian

Input: $[\rho, \theta_{ij}, \phi_k] = [7.071,\ 53.130°,\ 45°]^S$

Convert: $[\rho, \theta_{ij}, \phi_k] \rightarrow x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$ where $x = \rho \cdot \sin(\phi_k) \cdot \cos(\theta_{ij})$, $y = \rho \cdot \sin(\phi_k) \cdot \sin(\theta_{ij})$, $z = \rho \cdot \cos(\phi_k)$

$x = 7.071 \cdot \sin(45°) \cdot \cos(53.130°) = 7.071 \cdot 0.707 \cdot 0.6 = 3$ $y = 7.071 \cdot \sin(45°) \cdot \sin(53.130°) = 7.071 \cdot 0.707 \cdot 0.8 = 4$ $z = 7.071 \cdot \cos(45°) = 7.071 \cdot 0.707 = 5$

>> Output: [ijk] $\mathbf{r} = 3\mathbf{i} + 4\mathbf{j} + 5\mathbf{k} = [3\ 4\ 5]$

Rectangular to Polar

Input: $\mathbf{r} = 3\mathbf{i} + 4\mathbf{j} = [3\ 4]$

Convert: $[x, y] \rightarrow [r, \theta_{ij}]$ where $r = \sqrt{(x^2 + y^2)}$, $\theta_{ij} = \text{atan2}(y/x)$

$r = \sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$ $\theta_{ij} = \arctan(4/3) = 53.130°$ >> Output: $[r, \theta_{ij}] = [5\ 53.130°]^P$

Polar to Rectangular

Input: $[r, \theta_{ij}] = [5\ 53.130°]^P$

Convert: $[r, \theta_{ij}] \rightarrow x\mathbf{i} + y\mathbf{j}$ where $x = r \cdot \cos(\theta_{ij})$, $y = r \cdot \sin(\theta_{ij})$

$x = 5 \cdot \cos(53.130°) = 5 \cdot 0.6 = 3$ $y = 5 \cdot \sin(53.130°) = 5 \cdot 0.8 = 4$ >> Output: $\mathbf{r} = 3\mathbf{i} + 4\mathbf{j} = [3\ 4]$

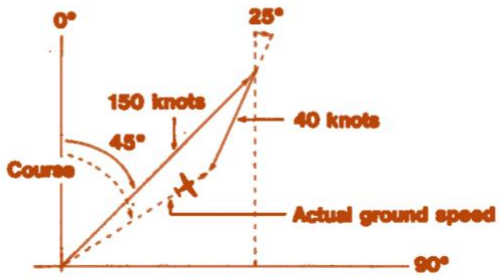
Practical 2D vector problem, from the HP25 Operating Manual, p70

Example: In his converted Swordfish aircraft, grizzled bush pilot Apeneck Sweeney reads an air speed of 150 knots and a heading of 045° from his instruments. The Swordfish is also being buffeted by a headwind of 40 knots from a bearing of 025°. What is the actual ground speed and course of the Swordfish?

Using R47, assuming default ADM = DEG. The example is displayed in SIG 3. Since we are referencing the same 90° x-axis as reference to all vectors, input, intermediate and output, we do not have to consider the 90° navigation offset from mathematical vector references:

150 ENTER 45 [DRG][yx≷$\bar{v}_2$] STO 'AirVel'	'AirVel': [150. 45.°]ᵖ
40 ENTER 25 [DRG][yx≷$\bar{v}_2$] STO 'Wind'	'Wind': [40. 25.°]ᵖ
- [→P] STO 'GndSpd'	'GndSpd': [113.24 51.94°]ᵖ

to give the vector of the actual ground speed and course. (For navigational convention, North is the x-axis.)



113.24	Actual ground speed of the Swordfish in knots.
51.94	Course of the Swordfish in degrees.

Practical 2D vector problem, from the HP-32E Operating Manual, p28

Example: Federation starship *Felicity* has emerged victorious from a furious battle with the starship *Thanatos* from the renegade planet Maldek. However, its automatic pilot is kaput, and its main thrust engine is locked on at 37.2 meganewtons directed along an angle of 25.2° from the star Ultima. Consulting the ship's star map, the navigator reports a hyperspace entrance vector of 51 meganewtons at an angle of 41.3° from Ultima. To what thrust and angle should the auxiliary engine be set, for *Felicity* to achieve alignment with the hyperspace entrance vector?

Using R47:

51 ENTER 41.3 [DRG][yx▸r̄v₂] STO 'HypSpc'

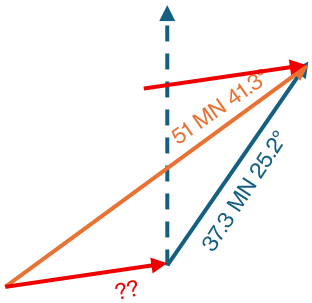
37.2 ENTER 25.2 [DRG][yx▸r̄v₂] STO 'MainEng'

- [→P] STO 'AuxPwr'

'HypSpc': [51. 41.3°]ᵖ

'MainEng': [37.3 25.2°]ᵖ

'AuxPwr': [18.419 0 75.361 3°]ᵖ MN from Ultima



18.4190 Convert to polar coordinates. Display shows required magnitude, in meganewtons, of auxiliary engine thrust vector.

75.3613 Required angle of auxiliary engine thrust vector.

Author Jaco Mostert